

Exploring IDoFT, iDFlakies, and NonDex

I analyzed the IDoFT dataset along with the iDFlakies and NonDex tools on a Kaggle notebook. iDFlakies detects order-dependent (OD) flaky tests. It reruns a test suite in randomized orders and flags a test as flaky if it passes in one order but fails in another. NonDex detects implementation-dependent (ID) flaky tests. It changes small Java behaviours that aren't actually guaranteed by the language, like the order you get back when you loop through a HashSet, and checks whether a test's result changes because of it. If the result changes, the test gets flagged as flaky.

IDoFT

IDoFT contains three CSVs: pr-data.csv for Java-Maven (8,075 records), gr-data.csv for Java-Gradle (1,029), and py-data.csv for Python (1,618). Java-Maven is dominated by ID records (64.3%), with OD a distant second at 13.3%. The Python dataset looks almost inverted: OD-Vic (49.7%) and OD-Brit (19.9%) dominate, and ID barely shows up at all (3 records, 0.2%). I didn't expect Java and Python to look this different. It makes me think either the real reasons tests get flaky are different between the two languages or it's just that flaky tests get labelled differently in each. Status fields mattered just as much for picking something I could actually reproduce. Flagging records with status keywords like irreproducible, deleted, archived, unmaintained, skipped, unavailable, invalid or failed caught 2,039 records total: 12.3% of Java-Maven, 17.5% of Java-Gradle, and a surprising 53.4% of Python, meaning more than half of it is effectively off the table for a direct reproduction attempt. Java-Maven spans 481 distinct projects, which gave me plenty of room to choose from. All of this fed into candidate selection, since a record being in the dataset doesn't automatically make it a convenient target to reproduce today. Whether the repository is still maintained, whether it's archived, and which exact historical commit got flagged all affect how practical it actually is to go back and reproduce a given test.

Candidate Selection

To narrow the IDoFT dataset to a small number of cases that were practical to reproduce and investigate in depth, I ranked Java-Maven projects by how many OD and ID records they had. Hadoop led OD by a wide margin, and NiFi came up strongly for ID, so those became my two picks. Within each project I only considered records with an Accepted status, since that means a maintainer had already confirmed the flaky-test report instead of it just sitting there as an open, unverified issue. Once I had a candidate, I cloned the repo and checked out its exact historical SHA to confirm the commit actually existed and matched what IDoFT listed before treating it as a real target.

	Hadoop case	NiFi case
Tool focus	iDFlakies(order dependence)	NonDex(API under-determinism)
Project	apache/hadoop	apache/nifi
IDoFT category	OD	ID

SHA	a585a73c3e02ac62350c136643a5e7f6095a3dbb	2bd752d868a8f3e36113b078bb576cf054e945e8
Test	org.apache.hadoop.hdfs.nfs.nfs3.TestRpcProgramNfs3.testEncryptedReadWrite	org.apache.nifi.authorization.CompositeUserGroupProviderTest.testMultipleProvidersWithCollaboratingUserGroupProvider

Observations

1. Historical build incompatibility

An initial IDoFT candidate (Mapper) could not be reproduced because the historical build inherited Java 1.6 compiler settings, while the available environment provided Java 17. A command-line attempt to override the source/target level did not resolve the effective compiler configuration. I treated this as a build and environment incompatibility, not a result about the test, and moved on to a different candidate.

2. Historical flaky label versus current stability

The IDoFT record for this test is an accepted OD case, but it passed every time I ran it under my current setup. Eight out of eight isolated runs passed, and all 22 tests in its class passed together as well, so the stable result wasn't just something that showed up because the test ran alone. That doesn't mean the test isn't flaky. It just means a flaky label from IDoFT isn't a guarantee the same failure will show up again under a different environment and at a different point in time.

3. Stable across repeats and the full class

Repeated isolated execution of the test produced an 8 out of 8 pass rate, with timing varying a bit between runs but no pass or fail instability showing up. Running it as part of its full 22-test class also passed cleanly, which is a useful check that the stable behaviour wasn't just something specific to running the test alone.

4. Randomized flaky detection can be expensive

The iDFlakies detector's randomized ordering strategy introduces substantial execution cost on a large project. A practical detector may need adaptive sampling, early stopping, project scoping, or other mechanisms to control runtime.

5. NonDex Test Selection Issue

NonDex reached the selected NiFi test and reported that it passed without shuffling. However, the overall Maven execution later ended with a “No tests were executed!” error. This made me suspect that the test-selection settings were not being handled correctly when the test was executed through the NonDex Maven plugin. Because of this,

I could not use the run to determine whether NonDex could reproduce the reported flaky behaviour.

6. Reproduction Depends on Both Historical Metadata and Execution Context

The IDoFT records provide useful reproduction information, including the project, commit SHA, test name, flaky category, and status. However, the observed behaviour can also depend on the execution environment and detection method. In my experiment, the accepted OD Hadoop case remained stable under my current setup. This suggests that historical metadata is important for identifying and reproducing a case, but it may not be sufficient to explain why the flaky behaviour originally occurred. Recording the build environment, execution strategy, and detector results alongside the historical metadata could make flaky-test reproduction and analysis more informative.

Limitations

1. IDoFT provides useful historical metadata, but reproducing older cases can be difficult when projects depend on outdated Java versions, dependencies, or build configurations.
2. iDFlakies can be expensive on larger projects. Its randomized test-ordering approach requires repeatedly executing tests, which can make detection impractical for large projects without mechanisms such as test selection, adaptive sampling, or early stopping.
3. The NonDex experiment showed that test selection and Maven/plugin configuration can affect whether the intended test is actually executed, making it important to verify the tool's execution output before interpreting its results.
4. iDFlakies and NonDex aren't interchangeable. iDFlakies looks for execution-order effects, while NonDex looks for tests that depend on Java API behaviour that isn't actually guaranteed. I picked one tool per test based on the category IDoFT had already assigned it, so I never checked whether the Hadoop test could also be flaky for NonDex-style reasons or whether the NiFi test could also be order dependent.

Research Direction

1. Investigate why historical flaky tests fail to reproduce, considering factors such as environment changes, tool failures, test-selection issues, and the absence of the original triggering condition.
2. Explore a diagnostic pipeline that distinguishes outcomes such as detected flaky behaviour, stable execution, non-reproducibility, and tool/build failure rather than treating them all as simply flaky or non-flaky.
3. Investigate adaptive sampling or early stopping to reduce the cost of randomized test-order detection on large projects.